Arithmetic Expression Geometry VI: Process Geometry

Native Programs, Evaluation Orders, and the Arithmetic–Process Equivalence

Mingli Yuan
Swarma Research
Beijing 100083, China

November 1, 2026
Draft manuscript under mathematical review
MSC 2020: 68Q55 (primary); 03G10, 05C05, 06A07, 18C50

Draft notice. This manuscript is a working draft, and it opens a slot in the series that the earlier governance files did not cover; its governing scope is `governance/00e-paper-VI-process-geometry-scope.md`. Here "draft" is an editorial and publication label, not a mathematical claim status, and it does not alter the stated status of any theorem, proposition, calibration, proposal, conjecture, or open problem.

Abstract

This paper opens the process side of arithmetic expression geometry. Papers 0–V take a sequence of arithmetic operations as their primitive history and develop its geometry; the present paper starts instead from an open finite program, which retains its holes, its sharing and discarding, its source and occurrence identity, and its construction order. Three coupled readings of one program process — construction, causal development, and frontier — are introduced, and an evaluation schedule is separated from the order it linearizes.

The organising principle is that the iota carrier of Paper V has arity one: acting at one slot, it induces an order on slots, and the spine and left/right comb orders of a sequential arithmetic expression are recovered as the single-spine case. On this basis a general evaluation-order theory is stated. For an arbitrary plane binary tree, among all linear extensions of the dependency order, the left-first postorder is the unique extension minimizing inversions against the corner order; the statement is proved for all trees and calibrated on all trees with three to six internal vertices (four to seven leaves).

The carousel of Pushkarev–Byzov is then related to the iota complex unit: its class-level four-cycle is a screw over an index, realized on the planar module $V_{\text{class}} \otimes V_{\text{index}}$ by $C^4 = S$, while the strong form "carousel $= J$" is refuted and the surviving correspondence is stated at exactly the strength established. The paper closes with a typed dictionary between arithmetic and process objects and with the equivalence statement the dictionary supports, separating what is proved, conditionally proved, finitely calibrated, proposed, and open. No theorem of Papers 0–V is redefined, strengthened, or weakened; the sequential theory appears as one leaf of the process theory.

Keywords: arithmetic expression geometry; process geometry; open program; construction order; evaluation schedule; linear extension; canonical evaluation order; carousel; iota; chirality; screw module; equivalence.

Contents

I Native processes and evaluation order **3**

1 Introduction and programme **3**
1.1 Two native formalisms . . . 3
1.2 What is established, and what this paper adds . . . 3
1.3 Status discipline . . . 4
1.4 Relation to the earlier papers . . . 4
1.5 Roadmap . . . 4

2 Program processes and the arity-one carrier **5**
2.1 The native object . . . 5
2.2 Three readings of one process . . . 5
2.3 The arity-one principle . . . 6
2.4 The sequential case . . . 6
2.5 The poset layer . . . 7

3 Evaluation orders and the canonical linear extension **7**
3.1 Plane trees, corner order, dependency order . . . 7
3.2 The canonical order . . . 8
3.3 Finite calibration and the limits of the statement . . . 9
3.4 The planar and poset layers . . . 9

II The carousel correspondence and the equivalence **9**

4 The carousel, the iota carrier, and the screw module **9**
4.1 The carousel of a plane tree . . . 9
4.2 The iota complex unit . . . 10
4.3 The index shift and the obstruction from the Laplacian . . . 10
4.4 The planar screw module . . . 11

5 The arithmetic–process dictionary and the equivalence **11**
5.1 What an equivalence must consist of . . . 11
5.2 The dictionary . . . 12
5.3 The equivalence statements . . . 12
5.4 Finite tests the equivalence must pass . . . 13

6 Claim boundaries and the remaining programme **13**
6.1 The claim ledger . . . 13
6.2 Open problems . . . 14
6.3 What this paper does not claim . . . 15
6.4 The remaining programme . . . 15

Part I
Native processes and evaluation order

1 Introduction and programme

1.1 Two native formalisms

Arithmetic expression geometry begins with a sequential arithmetic expression and keeps, rather than discards, its evaluation history. Paper I establishes the intrinsic classification of sequential trees, the marked spinal history, the chirality word, and the projective semantics those histories induce [6]. Paper 0 develops the geometry of the resulting spaces, and Paper V develops the six-state carrier on which three readings of one construction are compared [5, 7].

Process Geometry begins one level lower. Its native object is an *open finite program*: a typed expression DAG with ordered holes, explicit source and occurrence identity, explicit copy and discard, and retained construction history [1, 3]. Composition grafts an actual argument program into a hole; evaluation is one observation of the resulting process, not its identity. One process carries three coupled readings — construction K, causal development C, and frontier F — and the theory's recurring statement is negative: the same value, the same differential, or the same spectrum does not imply the same program.

The two formalisms meet at a single observation. The iota carrier of Paper V has *arity one*: it acts at one slot, and acting at one slot induces an order on slots. Once that order is available, the spine of a sequential arithmetic expression and the left and right combs that generate it are not primitive data but the single-spine case of a process order. This paper states the general theory that results and the equivalence with arithmetic expression geometry to which it leads.

1.2 What is established, and what this paper adds

Three bodies of work are imported and cited rather than reproved.

1. *The carrier.* Paper V defines the six-state carrier $W = \Lambda^1 E \oplus \Lambda^2 E$ with its three aspect–opposite-relation planes, the complex unit $\Omega = TSR$, and the chirality sign law [7]. The agreement of this carrier with the Adva iota frame is recorded in the carousel–iota overview note (note 40) as a correspondence; it is not a claim of Paper V's appendix. This paper uses the carrier as given and cites the correspondence where it uses it.

2. *The combinatorics.* Pushkarev–Byzov give the carousel of a plane binary tree, its five corner classes and the four-cycle that permutes them, and the order-two shadow on the corner counts [4]. Donaghey's transform between general and binary Catalan trees supplies the right/left bijections and the factorisation σ, θ, φ [2, 4].

3. *The process corpus.* Process Geometry supplies the native program, the three readings, cuts and slices, and their exact laws at declared revisions [1, 3].

On that basis the paper states:

- a definition of the program process and of an evaluation schedule as a linear extension of its construction order (§2);
- the arity-one principle, which recovers the spine and the comb orders from a slot order (§2);

- the evaluation-order theorem: for every plane binary tree, the left-first postorder is the unique linear extension of the dependency order minimizing inversions against the corner order (§3);
- the carousel correspondence: up to the non-canonical alignment of Proposition 4.2, the class-level four-cycle aligns with the four-cycle of the iota complex unit on the signed states of one plane; the count shadow is an involution; and both are realized on the planar screw module $V_{\text{class}} \otimes V_{\text{index}}$ by $C^4 = S$ (§4);
- the arithmetic–process dictionary and the equivalence statement it supports (§5), with the boundaries collected in §6.

1.3 Status discipline

Every statement carries one of the repository's status labels. In this paper the labels are used as follows, and the summary table of §6 records the assignment statement by statement.

Label	Used for
`PROVED`	a statement proved here or in a cited proof record
`CLASSICAL`	an imported theorem, with attribution
`COMPUTATIONALLY VERIFIED EXAMPLE`	a finite instance checked by a stored verifier
`STRUCTURAL PROPOSAL`	a reading or construction, not a theorem
`CONJECTURE`	a precise statement expecting proof or refutation
`OPEN`	a question with no settled answer

Three warnings apply throughout. First, a finite calibration is not a theorem. Second, an alignment that requires a choice is stated as non-canonical. Third, the equivalence of §5 is stated at the strength the dictionary supports; the directions it does not support are listed as open problems rather than as consequences.

1.4 Relation to the earlier papers

The dependency is one-way and is fixed by `governance/00e-paper-VI-process-geometry-scope.md`:

Paper	What this paper imports
Paper I	sequential histories, spine, chirality word, mirror and reversal
Paper 0	the expression-space language and the charge vocabulary
Paper V	the six-state carrier, Ω, and the chirality sign law

The paper proves no new statement inside the scope of those papers, and no proved node of Papers 0–V may depend on it. What it adds is the general process theory and the dictionary; what it does not do is replace the sequential theory. The sequential case is recovered as the single-spine leaf of the process theory, and the recovery is stated as a proposition rather than as a redefinition.

1.5 Roadmap

Section 2 fixes the native objects: the open program, the three readings, the schedule, the arity-one principle, and the spine and comb orders. Section 3 states and proves the canonical evaluation order and records its finite calibration. Section 4 develops the carousel correspondence and the screw carrier, including the refutation of the strong form. Section 5 gives the typed dictionary and the equivalence statements. Section 6 collects the claim ledger, the open problems, and the directions that would extend the paper.

2 Program processes and the arity-one carrier

2.1 The native object

Definition 2.1 (Open finite program)**.** An *open finite program* P consists of

1. a finite typed expression DAG $B(P)$, its *body*, whose nodes are operations and whose edges carry typed wires;
2. a finite set ∂P of *holes*, each a named position of declared type, which are the only open leaves of $B(P)$;
3. a labelling that keeps distinct, for every source in $B(P)$, its *occurrences*: sharing and discarding are explicit and are not quotiented away;
4. the construction history of $B(P)$: the nesting of scopes and the partial order in which nodes were introduced.

Composition grafts an argument program into a hole; the result is again an open finite program. Evaluation — numerical, symbolic, spectral — is an observation of such a program under a declared policy, not its identity. Consequently the implications

$$\text{same value} \Rightarrow \text{same program}, \qquad \text{same differential} \Rightarrow \text{same program}$$

both fail, and the failure is recorded rather than repaired by quotienting.

Definition 2.2 (Construction order)**.** The *construction order* of P is the partial order $\preceq_P$ on the operation nodes of $B(P)$ in which $u \preceq_P v$ when u must be constructed before v: dependency edges, scope nesting, and source/occurrence creation.

Definition 2.3 (Evaluation schedule)**.** An *evaluation schedule* of P is a linear extension of $\preceq_P$, that is, a total order σ on the operation nodes such that $u \preceq_P v$ implies u precedes v in σ. The set of schedules is written $\mathsf{Sch}(P)$.

Two schedules of one program may differ while the program does not. A schedule is therefore extra data; it is the process analogue of a chosen linear extension of a dependency poset, and it is not the construction order itself.

2.2 Three readings of one process

One program process is read in three coordinated ways.

- **Construction** K: which open programs are grafted into which holes, the nesting of scopes, explicit copy and discard, and the dependency partial order generated by those events.
- **Causal development** C: the orientation of construction from a completed past toward enabled future events; crossing an enabled node replaces the frontier wires it consumes by the wires it produces.
- **Frontier** F: a downward-closed set of completed events, read as the checked wires crossing from that past to its future — an occurrence-decorated open boundary, not a tuple of values.

The three readings are coupled by

$$\text{construction dependency} \longrightarrow \text{legal causal advance} \longrightarrow \text{new open cut},$$

and it is exactly this coupling that a schedule linearizes. The statement that K, C and F are readings of one process, rather than three structures, is the primitive of the theory; it is imported at the revisions the bibliography pins: the Adva technical report at `1b9bd090` and the Process Geometry mathematical core at `c47c96fa79123c677172278be59d67ca1cc891b1`; there the finite slices and their composition laws are checked [1, 3].

Remark 2.4 (Slices). For downward-closed $U \subseteq V$, the slice of P between the cuts U and V retains the events of $V \setminus U$, both cuts, the changed boundary wires, the unchanged through wires, the occurrences and the history. Adjacent slices compose after input revalidation, and two distinct linear schedules of one diamond yield the same outer slice. The last statement is the finite form of the separation between schedule and process: schedule path is extra data, not slice event order.

2.3 The arity-one principle

The link to arithmetic expression geometry is the carrier of Paper V. Its complex unit $\Omega = TSR$ acts on the six-state carrier $W = \Lambda^1 E \oplus \Lambda^2 E$; the three planes $W_T = \langle T, SR\rangle$, $W_S = \langle S, RT\rangle$, $W_R = \langle R, TS\rangle$ pair each reading with the oriented relation of the other two, and the chirality sign law states that cyclic permutations preserve Ω while transpositions reverse it [7].

Proposal 2.5 (Arity one)**.** The iota carrier acts at one slot. Reading a program's holes as the slots at which the carrier acts, the carrier induces on the holes the order in which they are consumed, and through that order an order on the operations that fill them. The spine of a sequential arithmetic expression, and the left and right combs that decompose a general expression tree, are then the single-spine and path cases of one slot order rather than independent primitives.

The proposal is structural: it names the mechanism by which the carrier reaches the combinatorics, and it is the organising principle of this paper. What it does not by itself assert is that every process order is realized by the carrier; that is the content of the equivalence statements of §5, each with its own status.

2.4 The sequential case

Definition 2.6 (Spine and combs)**.** A finite binary expression tree is *sequential* when its internal-node dependency poset has a unique linear extension; equivalently, when its internal vertices form a single spine [6]. In the marked spinal history of Paper I, the accumulator evolves along that spine and each step carries a one-hole context $C^{(1)}_{\omega,c}[z] = \omega(z, c)$ or $C^{(2)}_{\omega,c}[z] = \omega(c, z)$; the sequence of slot choices is the chirality word. A *left comb* is the spine whose contexts are all left slots, and a *right comb* the spine whose contexts are all right slots.

Proposition 2.7 (Sequential histories are single-spine programs)**.** *Every* free *marked spinal history of Paper I (one without an initial mark) is an open finite program with exactly one hole and a construction order whose unique linear extension is the history itself. Conversely, an open finite program with one hole whose body is a chain of unary one-hole contexts, with the slot convention of its contexts declared as data, is a marked spinal history.*

Proof. Immediate from the definitions: the body of the history is the chain of contexts, the single hole is the accumulator position, the construction order is the chain order, and the unique linear extension is the chronological order. The slot convention is exactly the chirality word. A bounded history $(x_0; \gamma)$ keeps the initial mark x_0 as an extra datum; the one-hole program encodes the context chain, not the mark, so the proposition is stated for free histories. The converse reads the

same data in the other direction; the two constructions are inverse once the slot labels are declared. Composition of chain programs is concatenation of their context chains, and it corresponds to chronological concatenation of histories. □

Remark 2.8 (What the sequential case does not see). A sequential history has one schedule, so the distinction between a process and its linearization is invisible there. Everything this paper adds begins with programs that have more than one schedule: branching bodies, multiple holes, and the interleavings that a general dependency order admits. In this sense Paper I is the single-leaf theory and the present paper is the tree-level one.

2.5 The poset layer

Fixing the sequential leaf does not yet fix the order theory of the branching case. For a program whose body is a tree with no sharing, the dependency poset is tree-shaped, and its linear extensions are counted by the tree's hook-length formula rather than by a single chain. A body with sharing has a DAG-shaped dependency poset, for which that count does not apply; the tree-level statements of §3 are therefore made only in the no-sharing case. Two independent structures must be kept apart:

- the *planar* structure, which records left and right slots and is carried by the corner order of §3;
- the *poset* structure, which records only dependency and forgets left/right, and is carried by the down-set lattice $O(P)$ of Paper I.

The distinction is not academic: the Donaghey-type transforms act on the planar structure and do not descend to the poset layer, a point developed in §3 and registered as an open direction in §6. Keeping the two layers apart is a standing requirement of this paper.

3 Evaluation orders and the canonical linear extension

3.1 Plane trees, corner order, dependency order

Definition 3.1 (Plane binary tree)**.** A *plane binary tree* is a rooted tree in which every internal vertex has a left and a right child, either of which may be a leaf; the children are ordered. Write $I(t)$ for the set of internal vertices, $n(t) = |I(t)|$, and $\ell(t) = n(t) + 1$ for the number of leaves. A vertex is a leaf when it has no children; every other vertex is internal and has a left and a right child position, either of which may lead to a leaf. Trees are drawn with the root at the top and the left child to the left.

The combinatorics of the carousel is stated for this class; the phrase *plane planted complete tree* used in the source is the same object once the root edge is contracted, and this paper keeps the plane binary presentation throughout.

Definition 3.2 (Corner order)**.** The *corner order* $P(t)$ is the preorder of $I(t)$: the root, then the left subtree, then the right subtree. It is the left standard traversal of the source: the vertex, then the left subtree, then the right subtree.

Definition 3.3 (Dependency order)**.** The *dependency order* on $I(t)$ is the descendant order: $u \preceq v$ when u lies in the subtree of v. A *linear extension* of t is a total order E of $I(t)$ in which every descendant precedes its ancestors; $\mathcal{L}(t)$ denotes the set of linear extensions.

A linear extension is exactly an evaluation schedule of the tree read as a program: children must be constructed before their parent, and independent subtrees may be interleaved arbitrarily. This is the special case of Definition 2.3 in which the program body is a tree.

3.2 The canonical order

Definition 3.4 (Left-first postorder)**.** The *left-first postorder* $Q(t)$ is the order: left subtree, then right subtree, then root. Equivalently, traverse the tree depth-first, always visiting the left child before the right, and list an internal vertex after both of its subtrees.

Definition 3.5 (Inversion count)**.** For $E \in \mathcal{L}(t)$ and the corner order $P(t)$,

$$\mathrm{inv}(E, P(t)) = \#\{(u, v) \ : \ u \text{ precedes } v \text{ in } P(t) \text{ and } v \text{ precedes } u \text{ in } E\}.$$

It counts the pairs on which the evaluation order departs from the corner order; it is the Kendall tau distance between the two orders restricted to pairs that the corner order orients.

Theorem 3.6 (Canonical evaluation order)**.** *For every plane binary tree t,*

$$Q(t) = \mathrm{argmin}_{E \in \mathcal{L}(t)} \, \mathrm{inv}(E, P(t)),$$

and the minimizer is unique.

Proof. Induction on $n(t)$. For $n(t) = 1$ the only extension is the root and the statement is immediate. Let $t = \mathrm{Node}(a, b)$ with $m = n(t)$, so that $I(t) = \{t\} \sqcup I(a) \sqcup I(b)$ and

$$P(t) = [t] + P(a) + P(b), \qquad Q(t) = Q(a) + Q(b) + [t].$$

Every $E \in \mathcal{L}(t)$ has the form $E = M + [t]$, where M is an interleaving of some $E_a \in \mathcal{L}(a)$ and $E_b \in \mathcal{L}(b)$: the root is last because it depends on both subtrees, and the restrictions of E to the two subtrees are linear extensions of them. Decompose

$$\mathrm{inv}(E, P(t)) = (m - 1) + \mathrm{inv}(E_a, P(a)) + \mathrm{inv}(E_b, P(b)) + \mathrm{cross}(M),$$

where the first term counts the root against every other vertex, and $\mathrm{cross}(M)$ counts the inversions between a vertex of a and a vertex of b: pairs that $P(t)$ orders a-before-b but M orders b-before-a.

Two observations finish the proof. First, $\mathrm{cross}(M) \geq 0$, with equality exactly when every vertex of a precedes every vertex of b in M, that is, when $M = E_a + E_b$ is a block. Second, by the induction hypothesis $\mathrm{inv}(E_a, P(a))$ is minimized uniquely at $Q(a)$ and $\mathrm{inv}(E_b, P(b))$ uniquely at $Q(b)$. Hence the right-hand side is minimized uniquely at $E_a = Q(a)$, $E_b = Q(b)$, and the block interleaving, that is, at $Q(a) + Q(b) + [t] = Q(t)$. □

Corollary 3.7 (The canonical schedule)**.** *Every plane binary tree has exactly one evaluation schedule that is closest to the corner order in the inversion metric, and it is the left-first postorder. In particular the choice of schedule across incomparable branches is not free once the criterion is fixed: it is forced.*

Remark 3.8 (Why the extremal criterion is the right one)*.* The corner order is the order in which the tree's boundary is walked; it is the only order the planar structure supplies without a choice. The canonical schedule is the linear extension that departs from it as little as the dependency order allows. The criterion is therefore not an optimisation chosen for convenience but the closest linearization of a planar order under a poset constraint; other criteria — for example preserving the left-to-right order of leaves — are recorded as open directions in §6 because they have not been shown to select the same extension.

3.3 Finite calibration and the limits of the statement

Calibration 3.9 (Canonical order, small trees)**.** For every plane binary tree with $3 \le n(t) \le 6$ internal vertices, equivalently $4 \le \ell(t) \le 7$ leaves (193 trees), the left-first postorder is the unique linear extension minimizing $\mathrm{inv}(\cdot, P(t))$. The check is stored at `research/iota-carousel/verify_canonical_choice.py`; the closed form of the calibration is the statement of Theorem 3.6, so the calibration is evidence for the implementation, not for the theorem.

Theorem 3.6 is a statement about plane binary trees and their descendant order. It does not by itself say that an arbitrary program's dependency order is tree-shaped, nor that a general program has a unique canonical schedule: a program DAG may admit incomparable events that no planar structure orders. What the theorem does supply is the canonical choice on the planar layer. The carousel's index is a different object: it is the source's traversal position over the vertex classes (the preorder over all classes), not the canonical postorder schedule; §4 states what is and is not established about the relation, and the relation is not an equality of orders.

3.4 The planar and poset layers

Two orders must be kept apart when this result is used.

- The *corner order* $P(t)$ is planar: it records left and right.
- The *down-set lattice* $O(Q)$ of a dependency poset Q records only dependency and forgets left and right; it is the lattice of Paper I, and its elements are down-sets rather than orderings of vertices.

The canonical order is a statement in the planar layer. Donaghey-type transforms act on the planar layer as well, and the question whether they descend to $O(Q)$ has a negative answer for the transform τ studied in the research notes (`OQ-098`); the exact statement is not needed here and is recorded in §6. The distinction matters for the equivalence of §5: a dictionary that identified the two layers would be wrong, and the equivalence is therefore stated with the planar data retained.

Part II
The carousel correspondence and the equivalence

4 The carousel, the iota carrier, and the screw module

4.1 The carousel of a plane tree

Pushkarev–Byzov study a transformation of plane planted trees and its action on the non-root vertices of the binary picture [4]. The vertices fall into five classes, written SR, LL, RL, RN, LN: SR is the son of the root, LL and RL are the left and right leaves, and LN and RN the left and right non-leaves. The source calls these classes *corners*; they are vertex classes, not geometric corners. Under the transformation the classes are transported cyclically at the class level,

$$LL \longrightarrow RL \longrightarrow RN \longrightarrow LN \longrightarrow LL,$$

with SR as the base corner and an index shift that records the traversal position; the counts of the five classes obey the closed transport law of the source (Theorem 4.3 and Corollary 4.4 of [4]).

Remark 4.1 (Two levels, and the one that is not a rotation). The class-level four-cycle and the count-level transport are different structures. On the constraint surface of the corner counts the transport is an *involution* of order two, not a four-cycle; the class-level cycle is invisible to the counts. This is the first reason the correspondence with the iota carrier must be stated carefully: an alignment at one level does not automatically descend to the other.

4.2 The iota complex unit

Paper V's carrier is $W = \Lambda^1 E \oplus \Lambda^2 E$ with basis T, S, R, SR, RT, TS and the three aspect–opposite-relation planes

$$W_T = \langle T, SR \rangle, \qquad W_S = \langle S, RT \rangle, \qquad W_R = \langle R, TS \rangle.$$

The generators satisfy $T^2 = S^2 = R^2 = 1$ and $\varepsilon_i \varepsilon_j + \varepsilon_j \varepsilon_i = 0$ for $i \neq j$, and $\Omega = TSR$ is central with $\Omega^2 = -1$; cyclic permutations preserve Ω and transpositions reverse it, the chirality sign law [7]. The complex unit J acts by rotation by a quarter turn inside each plane, sending a positive state to a signed state and preserving each plane.

Proposition 4.2 (A screened alignment)**.** *The strong statement "the carousel class-cycle is J on the six states" is false. The carousel has five classes with a four-cycle on four of them plus an entry from SR, while J is plane-preserving and moves a positive state to a signed one. A non-canonical alignment exists: the four-cycle can be matched to J's four-cycle on one plane in $3 \times 2 = 6$ ways (three planes, two orientations), but the SR entry and the index shift have no J counterpart.*

Proof record. The statement is the alignment result of the carousel–iota alignment note (note 32). The carrier identities $\Omega^2 = -I$, $\Omega^4 = I$ and plane preservation are replayed by `research/iota-carousel/verify_omega.py`; the six plane/orientation candidates are enumerated by hand in the alignment note, not by that script, and the absence of an SR/index counterpart is structural (it follows from the class list). The claim is recorded as a correspondence, not as a theorem of the carrier. The plane-preservation and sign action of J are the sign law of Paper V; the class list and cycle are the source's Theorem 4.1–4.3. The 3×2 count is exhaustive for the alignment of a four-cycle to a plane's signed four states. □

The proposition is deliberately negative: it fixes what the correspondence is *not*, and leaves the surviving positive statement to be stated at its own strength. That statement is a screw.

4.3 The index shift and the obstruction from the Laplacian

The carousel's index is a traversal position of the source over the vertex classes. The finite obstruction below uses the planar statistic $k(t) = \#\{\text{left leaves of } t\}$, which is the statistic the shift note and its verifier compute; whether the source's traversal index agrees with $k(t)$ on every tree is part of the correspondence and is not assumed here (Problem 6.2). A candidate for carrying k is the cut Laplacian: take the down-set lattice $O(P)$ of the dependency poset (the lattice of Paper I), its Hasse diagram $G(O(P))$, and the normalized graph Laplacian $H = L/(2d_{\max})$ on functions on its vertices. The candidate fails.

No-go 4.3 (The Laplacian does not carry the index)**.** The Hasse diagram of $O(P)$ forgets left and right, whereas $k(t)$ is planar: with $\ell = 4$ leaves (three internal vertices) four trees share the normalized spectrum (0, 0.1464, 0.5, 0.8536) yet carry different numbers of left leaves. Hence no function of this spectrum can recover k.

Proof record. The four trees and their spectra are listed in the shift note; the computation, including the down-set Hasse graph and the normalization $L/(2d_{\max})$, is replayed by `research/iota-carousel/verify_shift.py`. The argument is the planarity obstruction: the down-set lattice is a poset invariant, the index is not. □

A screw reading exists on a doubled construction: let $H_\iota = \mathrm{diag}(H, H)$ on two copies of the down-set space and $J_\iota = \left(\begin{smallmatrix} 0 & -I \\ I & 0 \end{smallmatrix}\right)$. Then $[J_\iota, H_\iota] = 0$, $J_\iota^2 = -I$, and with $A = -J_\iota H_\iota$ one has $A^2 = -H_\iota^2$ and $\exp(tA) = \cos(tH_\iota) - J_\iota \sin(tH_\iota)$, as replayed by `research/iota-carousel/verify_shift.py`. This is a construction on the doubled space, not a claim that the cut Laplacian itself carries the index. We therefore *propose* that the height be carried by a planar, unbounded structure (Problem 6.2).

4.4 The planar screw module

Construction 4.4 (Planar screw module)**.** Let V_{class} be the four-dimensional real space on the class-cycle $\{LL, RL, RN, LN\}$ and let V_{index} be the real space on the index set $\{e_k : k \geq 1\}$. Put $V = V_{\text{class}} \otimes V_{\text{index}}$ and define

$$\begin{aligned} J_{\text{cl}} &: (c,k) \longmapsto (c+1, k) && \text{(class part of the iota unit, order four)},\\ S &: (c,k) \longmapsto (c, k+1) && \text{(the shift/height)},\\ C &: (c,k) \longmapsto (c+1 \bmod 4,\ k + [c = LN]) && \text{(the carousel)}; \end{aligned}$$

Proposal 4.5 (The carousel is a screw)**.** On V the carousel satisfies $C^4 = S$, the class rotation satisfies $J_{\text{cl}}^4 = \mathrm{id}$, and $[J_{\text{cl}}, S] = 0$. The count shadow $M : (k, \ell, m, n) \mapsto (n+1, k, \ell - 1, m)$ on the constraint surface $k = m + 1$, $\ell = n + 1$ satisfies $M^2 = \mathrm{id}$, so the screw structure is invisible to the corner counts. The carousel is therefore a projective fourth root of the shift: the rotation half is supplied by the iota carrier, and the height half is an independent structure.

Calibration 4.6 (Finite check of the screw)**.** The relations $C^4 = S$, $J_{\text{cl}}^4 = \mathrm{id}$, $[J_{\text{cl}}, S] = 0$ and $M^2 = \mathrm{id}$ are exact identities of the definitions (one of four successive classes is LN); `research/iota-carousel/verify_planar.py` replays them on a finite range and reproduces the stored evidence `evidence-planar-v1.json`. The non-trivial step is the identification of this C with the source's carousel, which is the proposal above. Since S is a unilateral shift $e_k \mapsto e_{k+1}$, C is not invertible: the structure is a semigroup action, not a group action.

Remark 4.7 (What is proved and what is proposed). The existence of the module and the finite relations are `COMPUTATIONALLY VERIFIED EXAMPLE`. The reading of the module as *the* carrier of the carousel is a `STRUCTURAL PROPOSAL`: it is a choice of presentation, supported by the relations and by the refutation of the alternatives, but not forced by them. The paper keeps the two apart, and §6 records the residual questions: whether S can be derived from AEG/Adva data rather than postulated, whether SR can be absorbed into V, and whether the construction extends beyond the planar class.

5 The arithmetic–process dictionary and the equivalence

5.1 What an equivalence must consist of

An equivalence between arithmetic expression geometry and process geometry is not a single theorem. It consists of four layers, and each layer has its own strength:

1. *Objects*: an expression tree corresponds to a program body; a marked spinal history corresponds to a schedule; a chirality word corresponds to a slot order.
2. *Orders*: the dependency order of Paper I corresponds to the construction order of the process; the chronological order of a sequential history corresponds to the unique linear extension of a single-spine program.
3. *Carrier*: the six-state carrier of Paper V corresponds to the iota frame of the process corpus, plane by plane.
4. *Structure*: the arithmetic structures — one-hole contexts, projective evaluation, charge — correspond to process structures (graft traces, observations, cut linkage), or they do not, and the failure is recorded.

The paper states the dictionary at the first three layers and treats the fourth as a programme: the layers already established are listed with status, the layers that are only proposed are marked, and the layers that fail are recorded as boundaries rather than patched.

5.2 The dictionary

Arithmetic expression geometry	Process geometry	Status
expression tree (Paper I)	program body with ordered slots	`DEFINITION`
free marked spinal history (Paper I)	schedule of a single-spine program	Proposition 2.7, `PROVED`
chirality word (Paper I)	slot order induced by the arity-one carrier	`STRUCTURAL PROPOSAL`
spine and left/right combs (Paper I)	single-spine and path cases of a slot order	`STRUCTURAL PROPOSAL`
dependency poset (Paper I)	construction order (Definition 2.2)	`DEFINITION` (tree bodies)
corner order and linear extensions	schedules and their inversions	Theorem 3.6, `PROVED`
evaluation order of a sequential history	unique linear extension of a chain	`PROVED` (Proposition 2.7)
six-state carrier (Paper V)	iota frame of the process corpus	`CORRESPONDENCE` (note 40)
chirality sign law (Paper V)	sign action of J on the planes	`CORRESPONDENCE`
carousel class four-cycle	non-canonical alignment with J's four-cycle plus a shift	`STRUCTURAL PROPOSAL` + `CORRESPONDENCE`
carousel count shadow	involution $M^2 = \mathrm{id}$	`CORRESPONDENCE` (source) + calibration
one-hole contexts and projective evaluation	observations compiled from a program	`OPEN PROBLEM`
accumulative commutative charge	cut linkage / frontier data	`OPEN PROBLEM`

Remark 5.1 (Reading the table). The table is a typed dictionary, not a theorem. Each row asserts that a construction on one side has a counterpart on the other, at the strength named. The rows marked `OPEN` are directions, not identifications: no claim of this paper depends on them, and the equivalence statements below do not use them.

5.3 The equivalence statements

Theorem 5.2 (Sequential recovery)**.** *The single-spine fragment of the dictionary is an equivalence on objects: the map sending a* free *marked spinal history to its chain program and back is a bijection, compatible with concatenation of one-hole contexts, between free marked spinal histories with their chronological order and open finite programs with one hole and a chain body whose unary contexts carry the Paper I signature, with the slot convention declared as data. A bounded history keeps*

its initial mark as an extra datum, outside the bijection. Under it, the chirality word is the slot sequence, the unique schedule is the chronological order, and the spine is the body chain.

Proof. Both directions are the explicit construction of Proposition 2.7; compatibility is immediate because both sides compose by concatenation of contexts along the slot sequence, and the slot convention is the only choice involved. No statement is made about the branching case. □

Conjecture 5.3 (General process–expression equivalence)**.** There is a natural assignment from plane binary trees equipped with the corner order and dependency order to open finite programs equipped with a planar schedule structure — natural meaning compatible with subtree substitution on the two sides — which restricts to Theorem 5.2 on the single-spine fragment and sends the canonical evaluation order to the unique planar schedule closest to the corner order.

The conjecture is deliberately narrow: it asks for an assignment on the planar layer, not for an identification of arbitrary programs with expressions. What is presently known toward it is: the sequential fragment (Theorem 5.2); the canonical order on all plane trees (Theorem 3.6); the carrier identification (`CLASSICAL`); and the screw calibration (§4). What is missing is a general construction of the assignment and a proof that it preserves the schedule structure; both are recorded as open.

Proposal 5.4 (What the equivalence is for)**.** The purpose of the equivalence is to let one theory's established results be reused in the other without collapsing their objects. Concretely: an order-theoretic result about schedules becomes a statement about histories, and a geometric statement about the carrier becomes a statement about the process's chirality. It is not a licence to identify values with programs, schedules with processes, or charges with frontier data.

5.4 Finite tests the equivalence must pass

Three finite tests are stated so that later work can refute the conjecture rather than illustrate it.

1. *Tree census*: for every plane binary tree with at most six internal vertices (seven leaves), the dictionary image of the canonical order is the canonical schedule of the image program, and the inversion count is preserved. The tree side is already calibrated; the program side needs an executed fixture.
2. *Schedule count*: the number of schedules of the image of a tree equals the number of linear extensions of its dependency order, and the canonical schedule is the image of the canonical extension. The count is the tree's hook-length product; a mismatch is a refutation.
3. *Carrier check*: the dictionary's carrier map sends the sign law of Paper V to the sign action of J, including the transposition reversal; a mismatch at any of the three planes is a refutation.

None of the three tests is currently stored as an executed fixture on the program side; they are stated here as acceptance tests for the programme and are registered in §6. This is the honest status: the equivalence is proved on the sequential fragment, supported by the carrier identification and the carousel calibration, and conjectural in general.

6 Claim boundaries and the remaining programme

6.1 The claim ledger

The following table is the paper's ledger. It records the status of every substantive statement; the governing file is `governance/00e-paper-VI-process-geometry-scope.md`, and the status labels are those of `governance/05c-paper-VI-status-register.md`.

Statement	Status	Evidence
open finite program, construction order, schedule	`DEFINITION`	process corpus [1, 3]
three readings K, C, F of one process	`IMPORTED TECHNICAL`	process corpus
slice composition and schedule independence	`IMPORTED TECHNICAL`	process corpus, finite fixtures
arity-one principle (slot order from the carrier)	`STRUCTURAL PROPOSAL`	this paper, §2
free histories are single-spine programs	`PROVED`	Proposition 2.7
canonical evaluation order, all plane trees	`PROVED`	Theorem 3.6
canonical order, $3 \le n \le 6$ (193 trees)	`COMPUTATIONALLY VERIFIED EXAMPLE`	the canonical-choice verifier (§3)
screened alignment: carousel is not J	`PROVED` (correspondence record)	§4
Laplacian does not carry the index	`PROVED` (finite obstruction)	No-go 4.3
screw module $C^4 = S$	`COMPUTATIONALLY VERIFIED EXAMPLE`	`verify_planar.py`
carousel as *the* carrier of the screw	`STRUCTURAL PROPOSAL`	§4
canonical schedule (Corollary 3.7)	`PROVED`	Theorem 3.6
doubled screw construction $H_\iota = \mathrm{diag}(H, H)$	`CONSTRUCTION` + exact identities	`verify_shift.py`
sequential recovery equivalence	`PROVED`	Theorem 5.2
general process–expression equivalence	`CONJECTURE`	Conjecture 5.3
one-hole contexts as compiled observations	`OPEN`	§5
charge as frontier data	`OPEN`	§5

6.2 Open problems

Open problem 6.1 (The general assignment). Construct the assignment of Conjecture 5.3 and prove that it preserves the canonical order, or refute the conjecture by a tree whose schedule structure has no process image.

Open problem 6.2 (Derive the shift). Derive the index shift S from the arithmetic or process data rather than postulating it. The candidate carriers supplied by the cut Laplacian are excluded by No-go 4.3; the shift must come from a planar datum.

Open problem 6.3 (Absorb the base corner). Extend the planar module so that the base corner SR is represented rather than added as an entry, and check whether the extended module still satisfies $C^4 = S$.

Open problem 6.4 (Canonicality of the criterion). Decide whether another natural criterion — for example preserving the left-to-right order of leaves, or invariance under the mirror — selects the same unique linear extension as Theorem 3.6. Agreement would strengthen the canonicality; disagreement would show that the choice depends on the criterion and must be declared.

Open problem 6.5 (Two layers, one transform). State precisely the relation between the planar canonicalisation of Theorem 3.6 and the poset-layer obstruction for the transform τ (`OQ-098`): the first chooses one linear extension, the second says that τ does not descend to the down-set lattice. The two statements are different and the difference must be written down, not conflated.

Open problem 6.6 (Fringing and orbit growth). Decide whether the carousel effect, the source's fringing law and the orbit growth $O(n^{\log_2 3})$ correspond to arithmetic periodic orbits, as the research notes propose. This needs the arithmetic orbit structure and is not settled by the carrier correspondence.

Open problem 6.7 (Program-side fixtures). Store the three acceptance tests of §5 as executed fixtures on the program side: tree census, schedule count, and carrier check. Until they exist, the equivalence remains conjectural even where each side has been checked separately.

Open problem 6.8 (Presentation and objectification). Bring the process corpus's presentation and objectification layers into the paper's scope, or state explicitly that they are outside it. The present draft imports only the native process, the three readings, and the slice laws; the rest of the Process Geometry programme is cited, not developed.

6.3 What this paper does not claim

The paper does not claim that every program is an arithmetic expression, that values determine programs, that the equivalence is proved in general, that the carousel correspondence is canonical, that the screw module is the unique carrier, or that any complexity or decidability consequence follows from the correspondence. None of the proved statements of Papers 0–V is redefined, strengthened, or weakened here, and no proved node of those papers may depend on this one.

6.4 The remaining programme

The programme has a clear order of dependence. First the general assignment (Problem 6.1), because everything else is stated through it. Then the derivation of the shift (Problem 6.2) and the absorption of the base corner (Problem 6.3), which decide whether the planar module is the right carrier. Then the criterion question (Problem 6.4), which decides whether canonicality survives a change of presentation. The arithmetic directions — fringing and orbits (Problem 6.6) — come last, because they presuppose the arithmetic orbit structure and are not settled by the carrier alone.

The process corpus is cited at a declared revision. A later revision that changes the slice laws or the observer policy is a boundary of this paper, not an automatic extension of it: the paper states what it imported and when.

References

[1] Adva. The program-process core of adva: Native objects, three readings, and exact slice laws. `docs/TECHNICAL_REPORT_PROGRAM_PROCESS_CORE.md` in `mountain/adva` at commit `1b9bd090` (pin recorded in `research/process-representation-gap/reports/manifest-baseline.md`), 2026.

[2] R. Donaghey. Automorphisms on Catalan trees and bracketings. *Journal of Combinatorial Theory, Series B*, 29(1):75–90, 1980.

[3] Process Geometry. Mathematical core of process geometry. `docs/MATHEMATICAL_CORE.md` in `mountain/process-geometry` at commit `c47c96fa79123c677172278be59d67ca1cc891b1`, the revision pinned by `research/process-representation-gap/reports/manifest-baseline.md`, 2026.

[4] I. A. Pushkarev and V. A. Byzov. Donaghey's transformation: An elementary approach. *Journal of Mathematical Sciences*, 196(2):199–215, 2014. Translated from Zapiski Nauchnykh Seminarov POMI 411 (2013), 148–177.

[5] Mingli Yuan. Arithmetic expression geometry 0—arithmetic expression spaces: Expansion and motion, eikonal equation, horocycle pullbacks, commutativization, contact structure, and tearing. Mathematical-review manuscript, repository path `paper-0/`, 2026.

[6] Mingli Yuan. Arithmetic expression geometry i: Foundations—sequential histories, projective semantics, and zero geometry. Mathematical-review manuscript, repository path `paper-1/`, 2026.

[7] Mingli Yuan. Arithmetic expression geometry v: Triadic characteristics and threaded apertures. Draft manuscript, repository path `paper-5/`; governing scope `governance/00c-paper-V-triadic-aperture-scope.md`, 2026.